■ Kubernetes

Complete Study Guide

Session 1 · Fundamentals & Architecture

Pods · Deployments · ReplicaSets · Control Plane · kubectl

■ Audience	■■ Platform	■ Tool	■ Coverage
DevOps Beginner	Windows + WSL2	Kind (K8s in Docker)	Core Fundamentals

■ Table of Contents

1 · Environment Setup & Tools

System Information

Property	Value
OS	Ubuntu 24.04 LTS on WSL2
CPU	16 Cores
RAM	15 GB
Docker	Docker Desktop integrated with WSL
Prior Knowledge	Docker (already comfortable)

Tools Installed

Tool	Purpose	Verify Command	Version Use
kubectl	CLI to communicate with K8s clusters	kubectl version --client	v1.36.1
Kind	Kubernetes IN Docker – creates local clusters	kind version	v0.32.0

2 · Kubernetes Architecture

Kubernetes follows a **Master–Worker** architecture. All user commands flow through the **API Server**, which acts as the single entry point into the cluster.

High-Level Request Flow

```
User (DevOps Engineer)
     |
     v
kubectl  (CLI client)
     |
     v
API Server  <--- single entry point
     |
     v
   ETCD    (cluster state database)
```

Control Plane Components

Component	Role	Key Fact
API Server	Front door – all requests pass through here	Stateless; stores nothing itself
ETCD	Distributed key-value store for all cluster state	The 'source of truth'
Scheduler	Decides which Node a new Pod should run on	Considers resources & constraints
Controller Manager	Continuously reconciles desired vs actual state	Runs many controllers in one binary

Node-Level Components

Component	Role
Kubelet	Agent on every Node; ensures containers match spec
kube-proxy	Handles Service networking and load-balancing rules
KindNet	CNI plugin providing Pod-to-Pod network communication
CoreDNS	DNS server inside cluster – resolves service names to IPs

3 · Control Plane – Deep Dive

API Server

- Every kubectl command hits the API Server first.
- It validates requests and talks to ETCD.
- Example: kubectl get pods → API Server → ETCD

ETCD

- Stores: Pods, Deployments, Services, ConfigMaps, Secrets, Nodes, Namespaces.
- It is the cluster's single source of truth.
- Losing ETCD = losing all cluster knowledge.

Scheduler

- Watches for unscheduled Pods.
- Evaluates Node resources, taints, affinity rules.
- Assigns the best Node to each new Pod.

Controller Manager

- Bundles many controllers: ReplicaSet, Node, Job, Endpoint...
- Continuously loops: desired state vs actual state.
- If a Pod dies → triggers ReplicaSet to create a new one.

4 · Creating & Inspecting Your First Cluster

Step 1 – Create Cluster

```
$ kind create cluster --name dev-cluster
```

Step 2 – Verify

```
$ kubectl cluster-info
$ kubectl get nodes
```

Step 3 – Inspect All Components

```
$ kubectl get all -A
```

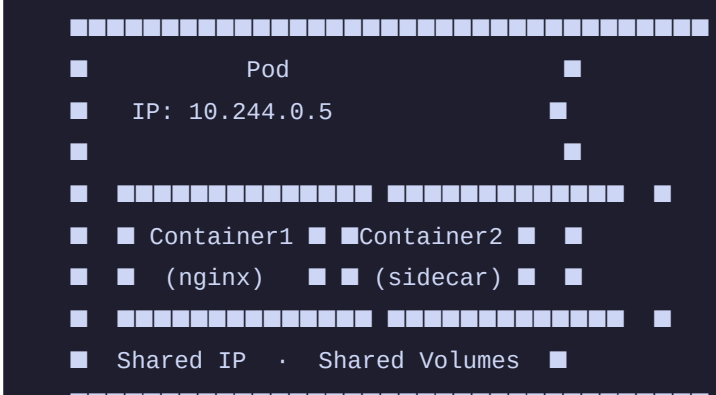
What You Will See

Component	Namespace	Purpose
CoreDNS	kube-system	DNS for services inside cluster
ETCD	kube-system	Cluster state storage
kube-apiserver	kube-system	API entry point
kube-scheduler	kube-system	Pod placement decisions
kube-controller-manager	kube-system	Reconciliation loops
kube-proxy	kube-system	Network rules per node
kindnet	kube-system	Pod-to-Pod CNI networking
local-path-provisioner	local-path-storage	Dynamic storage provisioning

5 · Pods – The Smallest Unit

A **Pod** is the smallest deployable unit in Kubernetes. It wraps one or more containers that share a network namespace (same IP) and storage volumes.

Pod Internals



Creating & Inspecting a Pod

```
$ # Create a Pod
$ kubectl run nginx --image=nginx

$ # List Pods
$ kubectl get pods

$ # Detailed inspection (Events, IP, Node)
$ kubectl describe pod nginx

$ # View container logs
$ kubectl logs nginx

$ # Interactive shell inside container
$ kubectl exec -it nginx -- bash
```

Pod Lifecycle Events

Event	Meaning
Scheduled	Scheduler picked a Node for the Pod
Pulling	Node is downloading the container image
Pulled	Image downloaded successfully
Created	Container created inside the Pod

Started	Container is now running

■ WHY STANDALONE PODS ARE BAD IN PRODUCTION

Standalone Pods have NO controller watching them. If a Pod crashes or the node fails, Kubernetes will NOT recreate it. Always use a Deployment (which manages a ReplicaSet) to ensure your Pods are self-healing.

6 · Deployments & ReplicaSets

Kubernetes Hierarchy

```
Deployment  (manages versions & rolling updates)
     |
     v
ReplicaSet  (maintains desired Pod count)
     |
     v
  Pod(s)    (runs your containers)

Real example:
nginx-deploy  -->  nginx-deploy-76c96b875b  -->  nginx-deploy-76c96b875b-sr5r7
```

Creating a Deployment

```
$ kubectl create deployment nginx-deploy --image=nginx

$ # Verify all layers
$ kubectl get deployments
$ kubectl get replicasets
$ kubectl get pods
```

Deployment vs ReplicaSet – Responsibilities

Deployment ■	ReplicaSet ■
Version management	Maintains desired Pod count
Rolling updates (zero-downtime)	Recreates crashed Pods (self-healing)
Rollbacks to previous version	Scales Pods up and down
Creates and owns ReplicaSets	Owned BY the Deployment

★ **KEY INSIGHT**

The Deployment creates the ReplicaSet. The ReplicaSet creates the Pods. When a Pod crashes, the ReplicaSet (not the Deployment) is responsible for creating the replacement. This is a common interview trick question!

7 · Scaling & Self-Healing

Scaling a Deployment

```
$ kubectl scale deployment nginx-deploy --replicas=3

$ # Confirm scale-out
$ kubectl get pods
$ kubectl get replicasets
```

Self-Healing Loop

```
Desired replicas = 3
Actual  replicas = 2   (one Pod crashed)

ReplicaSet notices:  3 - 2 = 1 missing
     |
     v
Creates new Pod automatically

Actual  replicas = 3   (restored!)
```

Demonstrate Self-Healing

```
$ # Delete one Pod manually
$ kubectl delete pod

$ # Watch Kubernetes recreate it instantly
$ kubectl get pods -w
```

✓ **SELF-HEALING EXPLAINED**

ReplicaSet continuously compares desired count with actual count. The moment a Pod disappears (crash, deletion, node failure), the ReplicaSet controller creates a replacement Pod automatically — no human intervention needed. This is one of Kubernetes' most powerful production features.

8 · kubectl Command Quick Reference

Command	Description	When to Use
`kind create cluster --name dev-cluster`	Create a local K8s cluster via Docker	Initial setup
`kubectl cluster-info`	Show control plane URL	Verify cluster is alive
`kubectl get nodes`	List all nodes in the cluster	Check node status
`kubectl get all -A`	List all resources in all namespaces	Full cluster overview
`kubectl run nginx --image=nginx`	Create a standalone Pod	Quick testing only
`kubectl get pods`	List Pods in current namespace	Check Pod status
`kubectl describe pod <name>`	Detailed Pod info + Events	Troubleshooting
`kubectl logs <name>`	Container stdout / stderr	Debug app errors
`kubectl exec -it <name> -- bash`	Shell into a container	Live debugging
`kubectl delete pod <name>`	Delete a specific Pod	Clean up / test self-heal
`kubectl create deployment ... --image=`	Create a managed Deployment	Production workloads
`kubectl get deployments`	List all Deployments	Check deploy status
`kubectl get replicasets`	List ReplicaSets	Verify replica management
`kubectl scale deployment <n> --replicas=`	Scale Pods up or down	Load management
`kubectl describe deployment <name>`	Detailed Deployment info	Troubleshoot deploys
`kubectl get pods -o wide`	Pods with Node and IP columns	Network debugging
`kubectl delete deployment <name>`	Delete Deployment and its Pods	Clean up

9 · Interview Questions & Answers

★ **INTERVIEW TIP**

These questions are commonly asked in DevOps / SRE / Platform Engineering interviews. Study the answers, but more importantly understand WHY each answer is correct.

Q1: Which component receives kubectl requests?

✓ The API Server. It is the single entry point for ALL interactions with the cluster.

Q2: Where does Kubernetes store cluster state?

✓ ETCD – a distributed key-value database. It stores Pods, Deployments, Services, ConfigMaps, Secrets, Nodes and Namespaces.

Q3: Which component decides where a Pod runs?

✓ The Scheduler. It evaluates available Nodes (CPU, memory, taints, affinity) and assigns the best one.

Q4: Who starts containers on a Node?

✓ Kubelet – the agent running on every Node. It reads the Pod spec and tells the container runtime to start the containers.

Q5: Who maintains the desired Pod count?

✓ ReplicaSet – NOT the Deployment. The Deployment creates the ReplicaSet; the ReplicaSet enforces the count.

Q6: Who manages rolling updates and rollbacks?

✓ The Deployment. It creates a new ReplicaSet for the new version and gradually shifts traffic while scaling down the old one.

Q7: Why are standalone Pods not used in production?

✓ Standalone Pods have no controller. If they crash, nobody recreates them. Always wrap Pods inside a Deployment.

Q8: A Pod is stuck in Pending. What is your first command?

✓ kubectl describe pod – the Events section reveals the root cause (e.g. insufficient resources, image pull errors, unschedulable).

Q9: What is Kubernetes self-healing?

✓ ReplicaSet continuously compares desired vs actual Pod count. If a Pod fails, it automatically creates a replacement without human intervention.

Q10: Who creates Pods during a scale-out?

✓ ReplicaSet. The kubectl scale command updates the Deployment spec; the Deployment updates the ReplicaSet's desired count; the ReplicaSet creates the Pods.

Q11: Explain the relationship: Deployment → ReplicaSet → Pod.

✓ Deployment manages application versions and rolling updates. It owns a ReplicaSet which maintains the desired Pod count. Each Pod runs the actual containers.

Q12: What does CoreDNS do?

✓ It provides DNS resolution inside the cluster, converting service names (e.g. backend-service.default.svc.cluster.local) into cluster IP addresses.

Q13: What is kube-proxy responsible for?

✓ It manages iptables/IPVS rules on each Node to route traffic destined for a Service to the correct backend Pods – enabling load balancing.